

Appunti di Basi di Dati

1. Algebra Relazionale

Differenza: "Non, Mai, Nessuno"

Trovagli studentiche **non** hanno mai sostenuto esami.

- Prendi gli studenti e togli quelli che hanno fatto almeno un esame → rimangono chi **non** ha fatto esami.

$R - S$

Quoziente: "Tutti, Ogni"

Trova i fan che sonostati in **tutte** le città.

- Prendi i fan, applica il quoziente → rimangono solo quelli che hanno visto **Tutte** le città.

A / B

A (Dividendo): Obiettivo della query (i fan) + "tutte le X" (le città)

B (Divisore): "Tutte le X richieste" (le città)

Trovare un MAX: "Più costoso, più..."

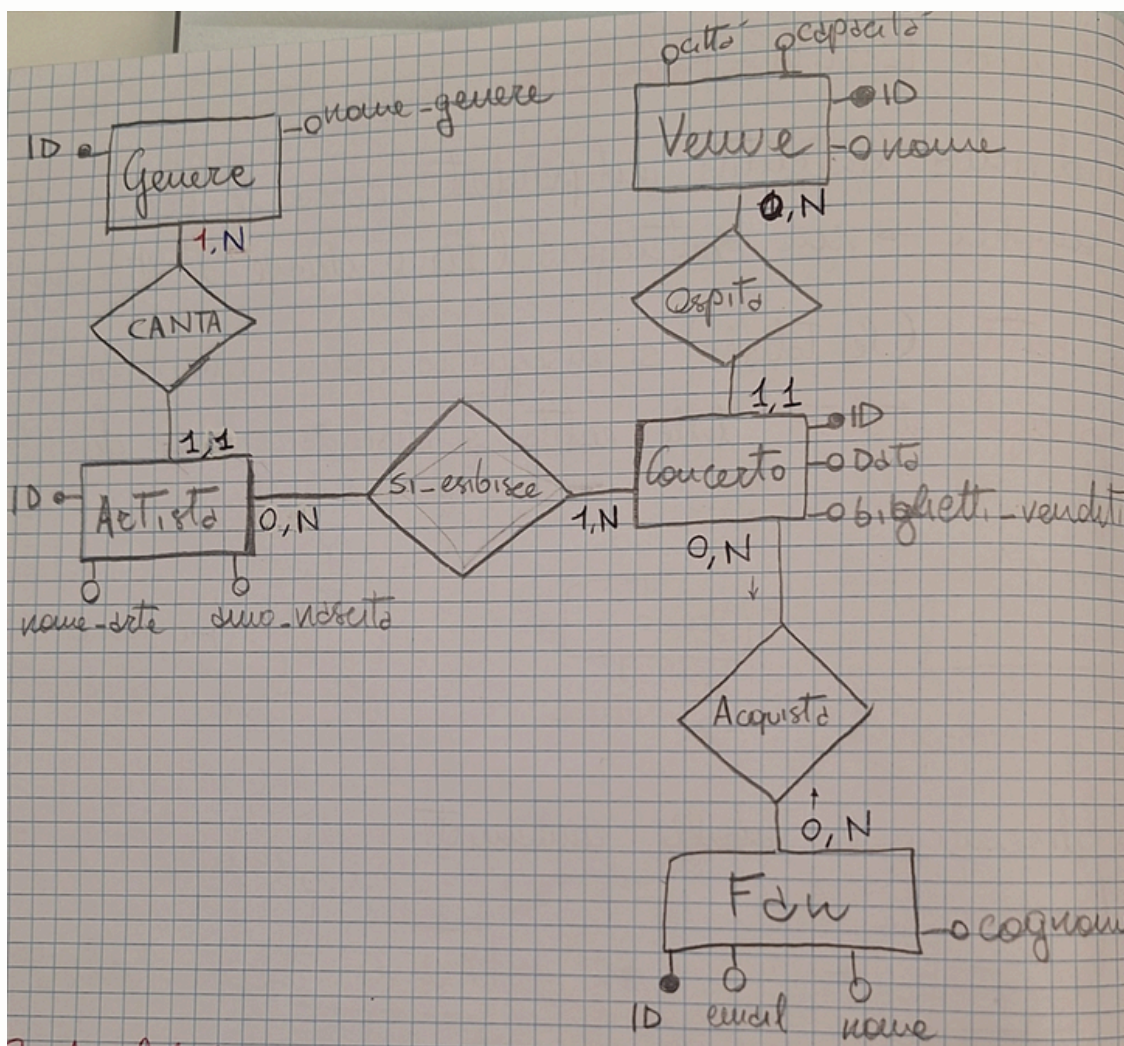
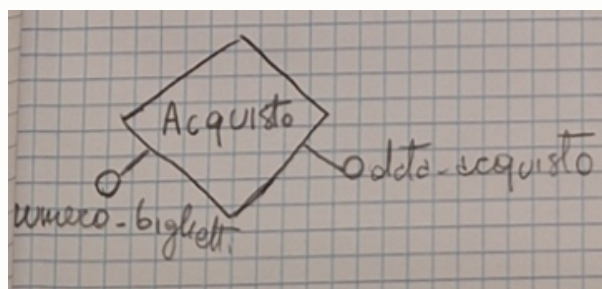
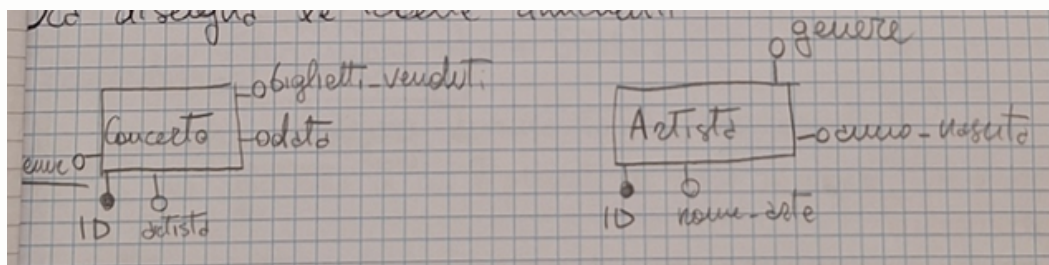
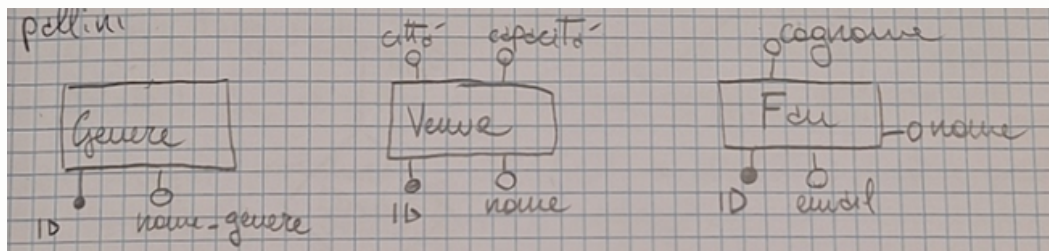
- Costruisci una tabella con i dati importanti (usa π) → $R1$
- Crea una copia di questa tabella usando la ridenominazione $p \rightarrow R2$
- Compara i dati che ti servono delle due tabelle con theta join. Solitamente con 2 criteri:
 - Stesso $X \rightarrow ID = ID_2$
 - Meno $Y \rightarrow Preventivo < Preventivo_2$

Questo forma $R3$.

- Adesso sottrai $R1 - R3$ per ottenere i "più..."

2. Schema Entità-Relazione (ER)

- Individuale entità indipendenti, cerca tabelle più semplici → non contengono chiavi di altre tabelle.



- Le **entità** (le Tabelle) si disegnano come rettangoli.
- I **singoli attributi** delle entità si disegnano come dei pallini. (Es. *Genere, Venue, Fan*).
- Per collegare due entità si utilizzano le **relazioni** (rappresentate da rombi), all'interno delle quali viene inserito un verbo che descrive il legame tra le entità nel contesto del dominio.
- Inoltre, le **chiavi esterne** non devono essere rappresentate come attributi (pallini) nelle entità, poiché il collegamento tra esse è già espresso dalla relazione stessa.
- Ora disegna le tabelle rimanenti (solitamente quelle con chiavi esterne).
- Le tabelle formate da unioni di chiavi sono **associazioni pure**. Si rappresentano come un rombo (es. *Acquisto*), con i suoi attributi (escluse le chiavi).

Esempio di Cardinalità

Dato un genere, quanti artisti può avere? Un genere può essere cantato da più artisti, quindi **1:N**.

Riepilogo dello Schema Grafico:

- **Genere** (ID, nome_genere) **1,N** — [CANTA] — **1,1Artista** (ID, nome_arte, anno_nascita)
- **Artista** **0,N** — [SI_ESIBISCE] — **1,NConcerto** (ID, Data, biglietti_venduti)
- **Concerto** **1,1** — [OSPITA] — **0,NVenue** (ID, nome, città, capacità)
- **Concerto** **0,N** — [ACQUISTA] — **0,NFan** (ID, email, nome, cognome)

3. SQL: Schemi di Base

- **Da dove prendo i dati?** **FROM** e **JOIN ... ON**
- **Chi scarto subito?** **WHERE** → C'è una condizione fissa sui dati base? Allora usa WHERE. MAI usare gli aggregati qui.
- **Come raggruppo?** **GROUP BY** → Cerca la dicitura "Per ogni..."
- **Quali "pacchetti" butto via?** **HAVING** → C'è un filtro sul Totale? (es. che hanno speso più di 50€, la cui media è superiore al Totale). Se il filtro richiede un aggregato, va messo qua.
- **Cosa stampo a schermo?** **SELECT** → attributi richiesti + quelli del GROUP BY e delle funzioni aggregate.

4. Pattern Frequenti negli Esami

Pattern 1: "Per Ogni" (Aggregazione + JOIN)

```
SELECT A.città, COUNT(*) B.repliche AS Tot_repliche
FROM Tabella1 A
JOIN Tabella2 B ON ...
GROUP BY A.città;
```

Pattern 2: "Il Massimo per Gruppo"

```
WITH Totali_Spesa AS (
    SELECT Supermercato, Cliente, SUM(Importo) AS Spesa_Totale
    FROM Scontrino
    GROUP BY Supermercato, Cliente
)
SELECT T1.Supermercato, T1.Cliente, T1.Spesa_Totale
FROM Totali_Spesa T1
WHERE T1.Spesa_Totale = (
    SELECT MAX(T2.Spesa_Totale)
    FROM Totali_Spesa T2
    WHERE T1.Supermercato = T2.Supermercato
);
```

Pattern 3: Confronto con il "Totale Globale" (HAVING + Subquery)

```
SELECT C.ID, C.Titolo, AVG(E.voto) AS Media_Corso
FROM Corso C
JOIN Esame E ON C.ID = E.id_corso
GROUP BY C.ID, C.Titolo
HAVING AVG(E.voto) > (
    SELECT AVG(voto)
    FROM Esame
);
```

Pattern 4: La negazione "Nessuno" / "MAI"

```
SELECT C.docente
FROM Corso C
JOIN Esame E ON C.ID = E.id_corso
GROUP BY C.ID, C.docente
HAVING MAX(E.voto) <= 22;
```

Pattern 5: 3+2 Confronto con Aggregato Locale

```
WITH SpesePazienti AS (
    SELECT P.CF, P.città, SUM(costo_tot) AS spese_tot
    FROM Paziente P
    JOIN Ricovero R ON P.CF = R.Paziente
    GROUP BY P.città, P.CF
)
SELECT Sp1.CF
FROM SpesePazienti Sp1
WHERE Sp1.spese_tot > 2 * (
    SELECT AVG(Sp2.spese_tot)
    FROM SpesePazienti Sp2
    WHERE Sp1.città = Sp2.città
);
```

5. Triggers

- **Cosafa scattare i trigger?** INSERT, UPDATE, DELETE.
- **Quando scatta?** BEFORE, AFTER, INSTEAD OF.
- **Quante volte scatta?** FOREACH ROW (per ogni riga) o FOR EACH STATEMENT (una volta per l'intero database).
- **Variabili:** NEW (nuovo record), OLD (vecchio record).

```

CREATE TRIGGER check_ricoveri
BEFORE INSERT ON Ricovero
FOR EACH ROW
BEGIN
    IF EXISTS (
        SELECT 1 FROM Ricovero WHERE paziente = NEW.paziente
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Impossibile aggiungere ricoveri';
    END IF;
END;

```

6. XQuery

Come navigo?

- Usa XPath, le barre / e //.
- Scrivendo //libro cerco una qualsiasi etichetta che si chiama <libro>.

Come lavora? Il ciclo FLWOR

Catena di montaggio composta da 5 stazioni, le 3 fondamentali sono: **FOR**, **LET**, **RETURN**.

- **Il FOR:** prende la pila di libri e li guarda uno ad uno.
- **Il LET:** mentre il for guarda il singolo libro, il let può fare dei calcoli su di esso. *"Mentre hai in mano questo libro contami quanti capitoli ha"*.
- In mezzo potrebbe anche esserci una condizione come **WHERE**.
- **Il RETURN:** per ogni giro che fa il FOR, il RETURN crea un nuovo pezzo di XML. È qui che scrivi i tag nuovi (es. <risultato>...</risultato>).

Assenza di GROUP BY

In XQuery, non esiste GROUP BY. Come risolvere? Es: *"Numero di libri scritti da OGNI autore"*.

- Per raggruppare, dobbiamo estrarre una lista di valori unici. Se dobbiamo fare un for su tutti i libri e ci sono 5 libri di "A", la query stamperà 5 volte "A".
- Quindi prendiamo gli autori e togliamo i doppi con distinct-values().

-- Generare un file xml che restituisce il numero di libri scritti da ogni autore

Passo 1 e 2: Creo una lista unica e la scorro

```
for $autore_corrente in distinct-values(doc("file.xml")//autore)
```

Passo 3: Conto i libri filtrando per l'autore che sto analizzando adesso

```
let $totale := count(doc("file.xml")//libro[autore=$autore_corrente])
```

```
return
```

```
<risultato>
```

```
  <nome>{$autore_corrente}</nome>
```

```
  <libri_scritti>{$totale}</libri_scritti>
```

```
</risultato>
```

I simboli di XQuery

- **Il dollaro :** \$a mia variabile. Ogni volta che vuoi creare una variabile temporanea oppure per leggere cosa c'è in quella variabile. Lo usi nel for, nel let per creare e nel return per stampare il contenuto.
- **Le Quadre []:** Il filtro. Si usano quando navighi con XPath. "Prendi questi elementi MA solo quelli che rispettano questa regola" (es. //libro[autore=...]).

7. Esercizio Ridondanza

- **Con ridondanza:** si tiene sempre aggiornato il valore.
- **Senza ridondanza:** non hai il valore aggiornato.
- Per calcolare la strategia che costa meno bisogna calcolare gli accessi di lettura (1 costo) e scrittura (2 costo) per entrambi i casi.

Esempio 1: Attributo "numero_biglietti_venduti"

Replica(..., numero_biglietti_venduti)

Prenotazione(replica, ...)

- i) Inserimento di una nuova prenotazione (5.000 volte al mese).
- ii) Controllo del numero di biglietti venduti per una replica (200 volte al mese).
- Volumi: 2.000 repliche, 100.000 prenotazioni. Essendo il numero delle prenotazioni per singola replica imprevedibile, dobbiamo fare una stima per "spalmare" i valori: $100.000 / 2.000 = 50$ (media prenotazioni per replica).

CON RIDONDANZA: Devi aggiornare numero_biglietti.

- Operazioni (i): scrivi prenotazione (+2), leggi tot numero_biglietti (+1), scrivi nuovo valore numero_biglietti (+2) = **+5** → Costo mensile = $5.000 \times 5 = 25.000$
- Operazioni (ii): lettura numero_biglietti (costo 1) → Costo mensile = $200 \times 1 = 200$.
- **Totale Con Ridondanza = 25.200**

SENZA RIDONDANZA (Senza Attributo):

- Operazioni (i): scrittura prenotazione (+2) → Costo = $5.000 \times 2 = 10.000$
- Operazioni (ii): il database calcola il dato al momento cercando tutte le prenotazioni associate a quella replica. Legge e conta uno per uno (50 letture medie). Costo = $200 \times 50 = 10.000$
- **Totale Senza Ridondanza = 20.000** → *Senza attributo conviene ($25.200 > 20.000$).*

Esempio 2: Esame 17-09-2025

Studente(ID, nome..., num_esami_superati) [ridondante]

Esame(ID, id_studente...)

- 20.000 studenti, 1.000 corsi, 50.000 esami.
- i) Inserimento di un Esame (200 volte).
- ii) Calcolo Tot esami_superati da studente (50 volte).
- Media esami per studente = $50.000 / 20.000 = 2,5$.

CON RIDONDANZA:

- (i) Scrittura esame (+2), lettura num_esami (+1), scrittura num_esami (+2) = +5. Totale: $200 \times 5 = 1.000$.
- (ii) Leggo num_esami (+1). Totale: $50 \times 1 = 50$.
- **Totale = 1.050**

SENZA RIDONDANZA:

- (i) Scrittura nuovo esame (+2). Totale: $200 \times 2 = 400$.
- (ii) Lettura esami associati (2,5 in media). Operazione = +2,5. Totale = $50 \times 2,5 = 125$.
- **Totale = 525** → *Senza ridondanza conviene ($1.050 > 525$).*

8. Esercizi Schemi R e Chiavi

a) Identificare le chiavi dello schema

Es: $R(A,B,C,D,E,F,G)$ con $F = \{A \rightarrow BC, CD \rightarrow E, E \rightarrow FG, G \rightarrow A\}$

1. Trovare $ND \rightarrow$ *Non a Destra* (quindi SOLO a sinistra) e mettilo in un insieme (non derivabile).
 $ND = \{D\}$.

- Con la chiusura di ND riesci a formare tutto R ? In questo caso no, perché con solo la D non riusciamo a ricavare le altre implicazioni.

2. A questo punto dobbiamo trovare il gruppo minimale $ND + SD$ per formare R . Aggiungiamo quindi 1 lettera presa da SD a ND .

- $SD = \text{Sinistra e Destra} = \{A, C, E, G\}$

- Scelgo $A \rightarrow \{D, A\}^+ = \{D, A, B, C, E, F, G\} \rightarrow$ Trovata 1 chiave!

- Scelgo $C \rightarrow \{D, C\}^+ = \{D, C, E, F, G, A, B\} \rightarrow$ Trovata 2 chiave! ◦ Scelgo E

- $\rightarrow \{D, E\}^+ = \{D, E, F, G, A, B, C\} \rightarrow$ Trovata 3 chiave! ◦ Scelgo $G \rightarrow \{D, G\}^+ =$

- $\{D, G, A, B, C, E, F\} \rightarrow$ Trovata 4 chiave!

b) Verifica 3NF e BCNF

Come deve essere uno schema per essere **3NF**? Per ogni dipendenza $X \rightarrow Y$:

- 1) X è una chiave intera (es. $\{DA\}, \{DC\}...$) OPPURE
- 2) Y è un attributo primo (cioè è una lettera contenuta in una chiave) $\rightarrow \{A, C, E, G\}$.

Verifica BCNF: Per ogni $X \rightarrow Y$, X è una chiave. Se la risposta è No, procedi con la scomposizione se richiesto.

Scomposizione Boyce-Codd (BCNF)

1. Prendi una dipendenza che viola le condizioni (es. $A \rightarrow BC$).
2. Prendi $R(A,B,C,D,E,F,G)$ e crea due nuove tabelle $R1$ ed $R2$.
3. In $R1$ metti tutti gli attributi della dipendenza: $R1 = \{A, B, C\}$.
4. In $R2$ metti R meno la parte destra della dipendenza: $R2 = R - \{B, C\} = \{A, D, E, F, G\}$.
5. Verifica le dipendenze rimanenti. (Attento alle transitività delle regole e se le dipendenze valgono ancora nei nuovi insiemi). Ripeti se necessario per $R3, R4...$

Scomposizione 3NF

Es: $R(A,B,C,D,E,F)$, Chiavi: $\{A,F\}$, $\{F,C\}$. $F = \{A \rightarrow B, C \rightarrow AD, AF \rightarrow EC\}$.

1. **Step 1)** Semplificare la parte dx delle dipendenze se hai 2 o più lettere che non rispettano la 3NF. (Es. $C \rightarrow AD$ diventa $C \rightarrow A$ e $C \rightarrow D$).
Nuovo insieme $F1 = \{A \rightarrow B, C \rightarrow A, C \rightarrow D, AF \rightarrow EC\}$.
2. **Step 2)** Creiamo tanti insiemi quante sono le relazioni. Se le lettere a sinistra della relazione sono identiche, fondiamo l'insieme.
 $R1 = \{A, B\}$, $R2 = \{C, A, D\}$, $R3 = \{A, F, E, C\}$.
3. **Step 3)** Minimo una delle tabelle prodotte deve contenere una chiave. $R3$ contiene AF , quindi abbiamo finito.

9. Esercizi Scheduler

Consideriamo lo schedule: $r_1(x) w_2(x) r_3(y) w_3(y) r_1(z) w_2(z) r_3(x)$

Come stabilire se lo schedule è CSR o VSR

1. **Metti in riga le variabili** (per ordine di apparizione, scrivi le operazioni):

◦ $X: r_1 \rightarrow w_2 \rightarrow r_3$

◦ $Y: r_3 \rightarrow w_1$

◦ $Z: r_2 \rightarrow w_2$

Attenzione! Le operazioni $r \rightarrow r$ NON diventano linee nel grafico! Non creano alcun conflitto.

2. **Disegna n cerchi per rappresentare i thread** ($T1, T2, T3$) su cui vengono eseguite le operazioni.
 - Ogni passaggio di stato verrà disegnato con una freccia. (Es. $r_1 \rightarrow w_2$ crea una freccia da $T1$ a $T2$).
 - Se si forma un ciclo (es. $T1 \rightarrow T2 \rightarrow T3 \rightarrow T1$), allora **NON È CSR!** (Se invece risulta CSR è in automatico anche VSR).

Come capire se lo schedule è VSR?

- **Step 1: La lettura iniziale.** Chi legge per primo il valore di una variabile e chi scrive subito dopo? Le letture devono avvenire prima che altre transazioni scrivano su quelle variabili.

◦ Per X : $T1$ è il primo a leggere. $T2$ successivamente scrive $\rightarrow T1 \rightarrow T2$.

Per Y : $T3$ è il primo a leggere. $T1$ successivamente scrive $\rightarrow T3 \rightarrow T1$.

- Per **Z**: Non c'è bisogno di controllarla perché viene "toccata" solo dal Thread 2, quindi nessuna regola di precedenza.
- **Step 2: Controllare le Reads-From.** Chi legge il dato appena scritto da qualcun altro?
 - Trova i casi in cui un Thread scrive una variabile e un altro la legge subito dopo. Chi scrive viene prima di chi legge.
 - Per **X**: T2 scrive, T3 legge quello che ha scritto T2 $\rightarrow T2 \rightarrow T3$.
- **Step 3: Controlla la Scrittura finale.** Chi scrive per ultimo nello schedule originale DEVE essere ultimo anche nell'ordine seriale.
 - Esempio: **Z**: $w2(z) \ r4(z) \ w1(z)$. L'ultima write è w1, quindi $T4 \rightarrow T1$.
- **Step 4: Metti insieme i vincoli.**
 - $T1 \rightarrow T2, T3 \rightarrow T1, T2 \rightarrow T3$.
 - Concatenando: $T1 \rightarrow T2 \rightarrow T3 \rightarrow T1$. Se si crea un paradosso o un ciclo, **NON è VSR**.

Trucchi per capire velocemente se lo schedule va in deadlock:

Il protocollo 2PL accetta SOLO schedule che sono CSR. Se due transazioni competono per una sola variabile in comune, è impossibile che vadano in deadlock (la terza si deve fare i "fatti suoi" però).

10. Calcolo Ricerca Puntuale

Dati di partenza e Spanned vs Unspanned

- **N** = record Totali
- **R** = dimensione dei record
- **B** = dimensione blocchi
- **V** = dimensione chiave
- **P** = dimensione puntatore

Se Unspanned: i record non possono essere spezzati tra due blocchi; se avanza spazio viene sprecato.

1. Step 1: Calcolo Fattore di Blocco (Bfr)

Quanti record entrano in un blocco? $Bfr = \lfloor B / R \rfloor$

2. Step 2: Calcolo numero totale di blocchi (b)

$$b = \lceil N / Bfr \rceil$$

Se Spanned: i record vengono spezzati, zero spazio sprecato.

1. Step 1: Calcolo numero totale di blocchi

$$b = \lceil (N \times R) / B \rceil$$

Costruire l'indice

Gli indici contengono la Chiave (V) e il Puntatore (P). Gli indici sono piccoli record unspanned.

- **Step 3: Calcola la dimensione del record indice**

$$R_i = V + P$$

- **Step 4: Calcola il fattore di blocco dell'indice**

$$B_{fr} = \lceil B / R_i \rceil$$

Risolvere i 3 possibili casi di ricerca dell'indice

- **Per indice Primario/Sparso:**

$$Accessi = \lceil \log_2 (\lceil b / B_{fr} \rceil) \rceil + 1$$

- **Per indice Secondario/Denso:**

$$Accessi = \lceil \log_2 (\lceil N / B_{fr} \rceil) \rceil + 1$$

- **Per indice Multilivello:** non ti servono logaritmi.

Prendi il numero di blocchi del livello 1, cioè b_1 , e continui a dividerlo per B_{fr} , fin quando il risultato è 1.

$$Accessi = \text{Numero livelli} + 1$$

Esempio: $b_1 = \lceil b / B_{fr} \rceil = X \rightarrow b_2 = \lceil X / B_{fr} \rceil = Y \rightarrow b_3 = \lceil Y / B_{fr} \rceil = 1 \rightarrow 3 \text{ livelli} + 1 = 4$ accessi.

Esercizio Pratico

$N = 1.500.000$ record, $R = 120$ byte, $B = 4096$ byte, $V = 16$ byte, $P = 8$ byte (Unspanned).

- **Step 1:** Fattore di blocco $B_{fr} = \lceil 4096 / 120 \rceil = 34$
- **Step 2:** Numero blocchi $b = \lceil 1.500.000 / 34 \rceil = 44118$ blocchi
- **Step 3:** Dim. record indice $R_i = 16 + 8 = 24$ byte
- **Step 4:** Fattore blocco indice $B_{fr_i} = \lceil 4096 / 24 \rceil = 170$

a) Ricerca su indice primario/sparso:

$$Accessi = \lceil \log_2(\lceil 44118 / 170 \rceil) \rceil + 1 = \lceil \log_2(260) \rceil + 1 = 9 + 1 = 10 \text{ accessi}$$

b) Ricerca su indice denso:

$$Accessi = \lceil \log_2(\lceil 1.500.000 / 170 \rceil) \rceil + 1 = \lceil \log_2(8824) \rceil + 1 = 14 + 1 = 15 \text{ accessi}$$

c) Ricerca su indice multilivello:

$$b_1 = \lceil 44118 / 170 \rceil = 260$$

$$b_2 = \lceil 260 / 170 \rceil = 2$$

$$b_3 = \lceil 2 / 170 \rceil = 1$$

Livelli = 3. $Accessi = 3 + 1$ (accesso al dato) = 4 accessi